数据结构与算法 · 绪论

> 主教材：李刚等《数据结构（C语言版）》微课第2版 · 人邮 | 参考：严蔚敏《数据结构》清华

学习目标

- 认识数据结构
- 分清数据·元素·项
- 程序 = 数据结构 + 算法
- 评价算法好坏的指标（时间复杂度）
- 用数据结构的视角分析一个真实的系统

第一部分 · 基本概念

1.1 为什么要学？仓库管理术

数据结构 = 数据的"仓库管理术"

想象你整理仓库：乱扔东西，找起来翻半天；分类贴标签，一秒拿到手。编程也一样，数据怎么存、怎么找，直接决定程序效率。数据结构就是研究"数据怎么存放和组织最有效率"的学问。

1.2 核心概念 · 学生档案

数据能输入计算机、能被处理的符号。照片、视频、文字……全是数据。 数据 = 一切信息
数据元素一个"完整个体"，比如一个学生、一条狗。是数据的基本单位。 元素 = 具体对象
数据项描述个体的细节：学号、姓名、年龄……最小信息单位。 项 = 细节信息

关系链： 数据（全集）→ 数据对象（同类集合）→ 数据元素（个体）→ 数据项（细节）

1.3 数据结构拆开看

结构 = 关系 + 排列方式。积木随意堆——没结构；按颜色分类——有结构。数据结构就是数据元素之间的"关系"和"组织方式"。

1.4 逻辑结构 vs 存储结构

逻辑结构 · 脑子里的关系

集合：全班点名册 线性：食堂排队 树形：校长→主任→老师 图形：微信好友网

存储结构 · 实际怎么放

顺序存储：连续排队，找第n个快，但插入麻烦。
链式存储：寻宝游戏，每站存下一站地址，插入删除灵活。

> 顺序 = 连续排队；链式 = 寻宝游戏

1.5 算法 · 烹饪步骤

算法 = 解决问题的具体步骤，就像菜谱。五个特点：有输入（0或多个）、有输出（0或多个）、有穷性、确定性、可行性。

> **程序 = 数据结构 + 算法**

数据结构 = 食材准备
算法 = 烹饪步骤

好算法 = 跑得快（时间复杂度）+ 省内存（空间复杂度）

> “跑得多快”到底怎么衡量？接着往下看，用高斯的故事讲清楚。

第二部分 · 算法好坏怎么衡量

1.6 高斯秒解 —— 老师，我算完了！

1.6.1 你听过这个故事吗？

1787年，德国一个小学课堂里，老师给全班同学出了一道题：

> **“从1加到100，等于多少？”**

同学们都掏出石板开始算：

- 1+2=3，3+3=6，6+4=10……
- 算着算着，有人算到50多了还在埋头苦算

结果不到**几秒钟**，一个10岁的小男孩举起石板说：

> **“老师，我算完了！答案是5050。”**

老师惊呆了——其他同学还在吭哧吭哧地算，他怎么这么快？

这个男孩就是后来被称为“数学王子”的**高斯**。

1.6.2 高斯是怎么算的？

高斯的思路其实很简单，他把1到100的数列“首尾配对”：

```
1 + 100 = 101
2 + 99  = 101
3 + 98  = 101
...
50 + 51 = 101
```

一共配了**50对**，每对的和都是**101**。

所以：

> **50 × 101 = 5050**

就这么两步，答案出来了。

1.6.3 这个故事的“编程版”理解

把这个故事翻译成编程语言：

普通同学的做法	高斯的做法
一个一个加：1+2+3+...+100	首尾配对，乘法一步到位
做了**99次加法运算**	只做了**1次乘法和1次加法**
数字越大，算得越慢	不管加到几万，都是两步搞定

> **这就是算法的力量：用聪明的办法，让计算机少干活、快出活。**

1.7 时间复杂度 —— 衡量算法“快不快”

1.7.1 什么是时间复杂度？

大白话：

> 时间复杂度就是“你的程序跑完需要多少步”，用来衡量一个算法**运行时间随数据量增长的趋势**。

不用管它具体跑了几毫秒，我们关心的是：

> **数据量变大时，你的程序会变慢多少？**

1.7.2 用高斯的故事理解时间复杂度

我们来比较两种算法的“步数”。

方法一：一个一个加（笨办法）

```
从1加到100：
第1步：1+2=3
第2步：3+3=6
第3步：6+4=10
...
第99步：算到100
```

如果加到 n，需要做 **(n-1) 次加法。**

n（加到几）	需要几步
100	99步
1000	999步
10000	9999步

> 数据量变成10倍，步数也变成10倍。这叫做 **O(n)** ——“线性增长”。

方法二：高斯公式（聪明办法）

公式是：**n × (n+1) ÷ 2**

不管 n 是100还是10000，都只需要 **3步**（1次乘法 + 1次加法 + 1次除法）。

n（加到几）	需要几步
100	3步
1000	3步
10000	3步

> 数据量变成100倍，步数还是不变。这叫做 **O(1)** ——“常数时间”。

1.7.3 看一眼就懂的时间复杂度对比表

算法	时间复杂度	加到100	加到10000	加到一亿	大白话解释
一个一个加	**O(n)**	99步	9999步	1亿步	数据越多，越慢
高斯公式	**O(1)**	3步	3步	3步	数据再多，速度不变

> **结论：高斯算法的时间复杂度是 O(1)，秒杀一个一个加的 O(n)。**

1.7.4 别忘了空间复杂度

时间复杂度看“跑得快不快”，**空间复杂度**看“占内存多不多”。好算法要两者兼顾：

> **好算法 = 跑得快（时间复杂度低）+ 省内存（空间复杂度低）**

（本阶段重点掌握时间复杂度，空间复杂度在后续章节逐步展开。）

1.8 练一练 · 时间复杂度小练习

练习1：判断时间复杂度

下面几个场景，你觉得哪个算法更快？用 O(1) 还是 O(n) 来判断。

① 在班级花名册里按学号找一个人

- 学号就是座位号，直接去那个位置找 → 不管班里多少人，都是一步到位。
- **答案：O(1)**（快！）

② 在没排序的书架上找一本《数据结构》

- 从左边一本一本翻到右边，运气好第一本就是，运气不好最后一本才是。书架上有 n 本书，最坏要翻 n 次。
- **答案：O(n)**（书越多越慢）

③ 用计算器算 999 × 888

- 直接按乘法键，一步出结果 → 不管数字多大，都是一步。
- **答案：O(1)**（快！）

④ 在微信通讯录里挨个翻找“张三”

- 从上往下滑，一个一个看名字，通讯录有 n 个人，最坏要翻 n 次。

- **答案：O(n)**（人越多越慢）

练习2：把生活中的事情分类

判断下面这些事，哪些是“一次搞定”（O(1)），哪些是“一个一个来”（O(n)）：

生活中的事	O(1) 还是 O(n)？	为什么？
用钥匙开自己家门	**O(1)**	一把钥匙对一把锁，直接开
在一堆钥匙里找家门钥匙	**O(n)**	一把一把试，运气不好试到最后一把
用遥控器按数字键换台	**O(1)**	按一下就到目标台
从第一台按顺序调到第50台	**O(n)**	按50下才到

练习3：看代码说时间复杂度

```
// 场景1：打印数组里的每个元素
for (int i = 0; i < n; i++) {
    printf("%d", arr[i]);
}
```

- 数组有 n 个数，就打印 n 次 → **O(n)**

```
// 场景2：直接取数组第5个元素
int x = arr[4];
```

- 不管数组多长，都是直接取 → **O(1)**

```
// 场景3：双重循环打印矩阵
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d", matrix[i][j]);
    }
}
```

- 外层 n 次，内层每次 n 次，总共 n×n = n^2 次 → **O(n^2)**（平方级，数据翻倍，时间变4倍）

解题方法 · 时间复杂度怎么分析（先看这里）

核心口诀：只看最高阶，忽略常数项，忽略底数，忽略系数

在开始之前，先记住三个原则：

1. **只关注循环执行次数最多的那段代码。**
2. **加法法则：** 总复杂度等于量级最大的那段代码的复杂度。（比如 O(n) + O(n^2)，结果取 O(n^2)）
3. **乘法法则：** 嵌套代码的复杂度等于嵌套内外代码复杂度的乘积。（比如外层 O(n)，内层 O(n)，总的就是 O(n ×

n) = O(n^2))

第一步：认识常见的时间复杂度量级（从快到慢）

- O(1)：常数阶（最快，不管数据多大，时间不变）
- O(log n)：对数阶（非常快，如二分查找）
- O(n)：线性阶（一般快，如遍历数组）
- O(n log n)：线性对数阶（一般，如快速排序、归并排序）
- O(n^2)：平方阶（较慢，如双重循环）
- O(2^n)、O(n!)：指数阶、阶乘阶（极慢，通常不讨论，遇到这种算法基本会超时）

第二步：拿到一段代码，按下面 3 步走

1. **找循环：** 看看有没有循环结构。如果没有循环，基本就是 O(1)。
2. **找规律：** 确定循环变量是怎么变化的。
    - 每次加 1，减 1？→ 可能是 O(n) 或 O(n^2)。
    - 每次乘 2，除 2？→ 肯定是 O(log n) 相关。
    - 循环次数依赖外层变量？→ 尝试用等差数列求和公式算一算。
3. **化简：** 把算出来的表达式（比如 $n^2/2 + n$）拿出来，只保留最高次幂，去掉系数，就是最终答案。

第三部分 · 综合案例：教学评价系统的数据结构视角

> 前面认识了“系统有哪几大模块”，这一节我们**换个视角**——不只看系统“长什么样”，更要看系统**内部的数据是怎么组织、怎么存储、怎么关联的**。这就是**数据结构的视角**。

1.9 为什么系统设计要看数据结构？

打个比方你就懂了

盖一栋楼：

- **外观设计**（模块划分）：哪儿是教室、哪儿是食堂、哪儿是宿舍 → 这是“功能模块”
- **内部结构**（数据结构）：墙里的钢筋怎么布、水管怎么走、电线怎么拉 → 这是“数据结构”

外观决定好不好用，内部结构决定牢不牢、稳不稳、出问题了好不好修。

做系统也一样：

- **模块划分**决定功能清不清晰
- **数据结构**决定系统跑得快不快、数据乱不乱、以后好不好改

> **一句话记住：模块是“骨架”，数据结构是“血脉”。**

第四部分 · 本章小结 & 想一想

1.16 本章小结

数据结构 = 数据怎么组织存储；
数据→元素→项；逻辑/存储结构；
算法评价：时间复杂度 + 空间复杂度；
真实系统里，模块是“骨架”，数据结构是“血脉”。

本章知识地图：

层次	内容
基本概念	数据 → 数据对象 → 数据元素 → 数据项；逻辑结构 / 存储结构
算法	程序 = 数据结构 + 算法；五大特性
算法评价	时间复杂度（O(1)、O(n)、$O(n^2)$）、空间复杂度
综合应用	教学评价系统：集合 / 树形 / 线性 / 图形 + 五张核心表

1.17 想一想

① 怎么跟同学解释“数据结构是干什么的”？
② 班级花名册是数据、元素还是项？
③ 手机通讯录是什么逻辑结构？
④ 为什么说“好程序 = 好结构 + 好算法”？

1.17.1 怎么跟同学解释“数据结构是干什么的”？

简单回答：

> “数据结构就是教你怎么‘收拾东西’的学问。就像你整理房间一样——衣服挂衣柜、书放书架、袜子放抽屉，找的时候才快。程序处理数据也是这个理，数据怎么存、怎么摆、怎么找，就是数据结构研究的事。”

更简单的说法：

> “数据结构 = 数据的‘收纳术’ + ‘寻宝图’。”

1.17.2 班级花名册是数据、元素还是项？

答案： 要看你怎么看它。

角度	属于什么	理由
整本花名册（所有学生信息）	**数据**	它是一个完整的信息集合，能输入计算机处理
花名册里的**某一个学生**（比如“张三”）	**数据元素**	它是一个独立、完整的个体

角度	属于什么	理由
花名册里某个学生的**具体信息**（比如“张三的学号”“张三的年龄”）	**数据项**	它是描述一个对象的最小细节，不能再分了

一句话总结：

> 花名册是**数据**，里面的每个人是**数据元素**，每个人的姓名/学号是**数据项**。

1.17.3 手机通讯录是什么逻辑结构？

答案： 线性结构（更准确地说是**线性表**）。

为什么？

打开手机通讯录，联系人是一个挨着一个排列的：

- 第1个是“阿花”，第2个是“阿明”，第3个是“阿杰”……
- 除了第一个和最后一个，每个人都有一个“上一个”和一个“下一个”

这就像**排队**——大家前后有序，一个接一个。

> **注意**：虽然通讯录有字母索引（A、B、C…），可以快速跳转，但存储和展示的基本逻辑仍然是**线性顺序**。字母索引只是加快查找的“快捷方式”，不改变它本质上是线性结构的事实。

那什么情况下会变成树形或图形结构呢？

- 如果你把“家人”“同事”“同学”分组，组里再分人 → **树形结构**
- 如果加上“谁是谁的介绍人”“谁和谁是好友”这种关系 → **图形结构**

但通讯录最基础的呈现方式，就是**线性结构**。

1.17.4 为什么说“好程序 = 好结构 + 好算法”？

> “做一个好程序，就像开一家好餐厅。**数据结构**是‘食材怎么存、怎么摆’——冰箱怎么分区、货架怎么排，取用才快。**算法**是‘菜怎么做’——先洗再切还是先切再洗，步骤不同，效率天差地别。食材摆得乱，厨师找半天（结构差）；步骤设计蠢，一道菜做一小时（算法差）。两者都做好，才能又快又好地出菜。”

用技术语言说：

- **好结构** → 数据存取快、占用内存少、方便增删改查
- **好算法** → 处理数据快、步骤少、不浪费计算资源

缺一不可：

- 结构好 + 算法差 → 找数据快，但处理慢（白搭）
- 结构差 + 算法好 → 处理快，但找数据慢（卡在第一步）
- 结构好 + 算法好 → 又快又省，程序才“优秀”

> **一句话记住：结构决定“找得快不快”，算法决定“做得快不快”，两个都好，程序才顶呱呱。**

【额外小彩蛋 —— 方便你记忆】

把这四个问题的答案串成口诀：

> **结构是仓库管理术，**（①）
> **花名册里分三层：**（②）
> **通讯录像排排队，**（③）
> **结构算法两手抓，程序才能顶呱呱！**（④）